

Task-Driven Object Detection:
Accelerating MobileNetV3 and GGNNs
on the VEGA RISC-V Architecture

Maneth Perera, Dulana Pitiwaduge and Himel Perera

ArchiMIDs, 157

University of Moratuwa

Table of Contents

Abstract.....	4
I. Introduction	5
II. Background Research.....	6
III. Goal and Objectives	7
IV. Design Process	8
i. Problem Statement.....	8
ii. Functional Specification.....	9
iii. Proposed Design.....	9
iv. Analysis of Final Design.....	11
v. Testing and Implementation of Final Design.....	12
V. Results and Discussion.....	13
VI. Conclusion	14
VII. References.....	14

List of Tables

Table	Caption	Page No
1	Comparison of Original Implementation with Implementation using MobileNetV3 with R-CNN	12

List Of Figures

Figure	Caption	Page no
1	Pipelines of original Resnet101 based architecture and proposed MobileNetV3 based architecture	9
2	Original Implementation with R-CNN	12
3	Implementation using MobileNetV3 with R-CNN	12

Abstract

This report presents a hardware-software co-design strategy to deploy a task-driven object detection framework on the Genesys-2 FPGA, overcoming the severe memory and computational bottlenecks typical of edge AI applications. Conventional task-aware models rely on heavy feature extractors like ResNet-101 paired with Gated Graph Neural Networks , which easily exceed the Block RAM (BRAM) and DSP slice limits of embedded hardware. To achieve real-time inference, the ResNet-101 backbone was replaced with a MobileNet V3 Large architecture. By leveraging depthwise separable convolutions, this optimization reduced the parameter count by over 85% and decreased the inference computational load.

The system integrates an indigenous 64-bit VEGA RISC-V soft-core processor to handle system control, text-prompt parsing, and direct memory access. To mitigate the Von Neumann memory bottleneck, the GGNN's matrix operations were offloaded to a custom hardware accelerator utilizing a 2D Systolic Array. This array employs matrix tiling and a Weight-Stationary dataflow, caching intermediate feature maps directly in the FPGA's shared scratchpad memory (BRAM). Experimental results validate the architectural shift: the MobileNet-optimized pipeline reduced the model while achieving a comparable mean Average Precision to the original baseline , proving its viability as a secure, low-latency blueprint for sovereign edge AI computing.

I. Introduction

The Core Problem: From Task-Agnostic to Task-Driven Vision Conventional object detection algorithms are fundamentally task-agnostic. They expend valuable compute resources identifying and classifying every object within a scene, regardless of its relevance to the user's actual operational goal. For example, an autonomous system tasked with "extinguishing a fire" does not need high-resolution feature maps of distant background furniture. In edge-computing scenarios, this inability to filter context leads to severe computational inefficiency and reduced decision relevance. The foundational problem, therefore, is the need for a **task-driven object detection framework** that bridges this semantic gap by dynamically prioritizing objects based on specific, text-based operational prompts.

The Hardware Deployment Challenge : While theoretical AI models such as Gated Graph Neural Networks paired with deep feature extractors have been proposed to solve task-aware detection, deploying them on edge devices introduces critical engineering bottlenecks. High-accuracy backbones like ResNet-101 contain tens of millions of parameters. When deployed on embedded FPGAs, these massive models immediately hit the bottleneck, saturating external memory bandwidth and overwhelming the limited on-chip Block RAM (BRAM) and logic resources. Addressing the problem statement effectively requires more than porting software, it demands a deep hardware-software co-design strategy that optimizes the neural network architecture specifically for the silicon it runs on.

To solve this, the design integrates the indigenous VEGA RISC-V processor for system control and I/O alongside a custom hardware accelerator. To satisfy strict edge resource constraints, the traditional compute-heavy backbone is replaced with a MobileNet V3 Large architecture, utilizing depthwise separable convolutions to drastically reduce parameter count. The GGNN context modeling is then accelerated using a custom, resource-optimized systolic array mapped directly to the FPGA's DSP slices.

Significance of the Work:

1. **Algorithmic Efficiency:** It pushes the boundaries of edge AI by enabling embedded systems to understand object *utility* based on human-centric tasks, rather than mere object identity.

Design Contest Stage 1 Report

2. **Hardware Innovation:** It demonstrates how severe hardware constraints (memory bandwidth and DSP limits) can be successfully overcome through aggressive network optimization (MobileNet V3 substitution) and tailored dataflow architectures (Weight Stationary systolic arrays).

II. Background Research

The proposed task-aware detection pipeline relies on a heterogeneous hardware-software co-design utilizing the Genesys 2 FPGA and the VEGA RISC-V processor. We utilize the 64-bit VEGA AS-series featuring a 16-stage Out-of-Order execution pipeline. Utilizing RISC-V standard extensions, the VEGA core efficiently handles system control, text-prompt parsing, and DMA memory transfers.

The Accelerator Fabric : The FPGA provides 840 DSP48E1 slices and 16 Mbits of Block RAM (BRAM). These DSP slices are the primary engine for the high-speed Multiply-Accumulate (MAC) operations required by the Neural Network , allowing us to build a custom Verilog systolic array alongside the 50,950 logic slices used for the soft-core.

A primary challenge in FPGA neural network deployment is the "Von Neumann Bottleneck " , the high latency of fetching massive weight matrices from the external 1GB DDR3 memory.

The ResNet Limitation: The original CVPR 2019 framework uses a ResNet-101 backbone. Its ~44.5 million parameters rapidly saturate the memory interconnect, starving the DSP slices and throttling inference speed.

The MobileNet V3 Optimization: To achieve real-time edge performance, we replace ResNet-101 with MobileNet V3 Large. By utilizing Depthwise Separable Convolutions, the parameter count drops by >85%. This allows the custom accelerator to cache a vast majority of the active weights and feature maps directly inside the 16 Mb of internal BRAM, enabling a highly efficient Weight Stationary dataflow that maximizes DSP utilization.

While large language/vision models (e.g., Gemini) can process text and images, relying on cloud APIs is unsuitable for this specific deployment for three reasons:

Deterministic Latency: FPGAs process data through dedicated physical hardware paths, offering highly deterministic, real-time execution that cloud-based models cannot guarantee due to network latency.

Design Contest Stage 1 Report

Power & Compute Efficiency: General-purpose GPUs used for cloud LLMs consume massive amounts of power. The FPGA offers a significantly higher Performance-per-Watt ratio specifically tuned for edge inference.

III. Goal and Objectives

To design and deploy a real-time, edge-optimized task-driven object detection framework on the Genesys-2 FPGA platform. This system will overcome traditional memory bandwidth bottlenecks by utilizing a hardware-software co-design, integrating the indigenous VEGA RISC-V processor with a custom neural network accelerator to intelligently identify objects based on semantic user prompts.

Specific Objectives

To successfully accomplish this goal, the project will execute the following objectives:

1. **Algorithmic Optimization:** Replace the computationally heavy ResNet-101 backbone with a lightweight MobileNet V3 architecture. This will reduce the model's parameter count by over 85%, allowing it to fit within edge memory constraints while preserving the Gated Graph Neural Network for task-aware context modeling.
2. **Hardware Accelerator Design:** Develop a custom systolic array mapped to the FPGA's DSP48E1 slices and Block RAM . This will facilitate a Weight Stationary dataflow to execute massive matrix multiplications with ultra-low latency.
3. **System on Chip Integration:** Integrate the custom hardware accelerator with the 64-bit VEGA RISC-V soft-core processor using high-bandwidth AXI interconnects. The VEGA core will be dedicated to system control, task-prompt parsing, and DMA memory transfers.
4. **Edge Deployment & Validation:** Deploy the complete pipeline onto the physical Genesys-2 board and validate its end-to-end inference accuracy and real-time performance against the 14 defined tasks of the COCO-Tasks dataset.

IV. Design Process

i. Problem Statement

To design and deploy an edge-optimized, task-driven object detection framework that identifies the most suitable object in an image, based on a specific task that is given by a text prompt.

Core Deliverables;

- An intelligent pipeline utilizing a lightweight model to filter and prioritize object detections based on user defined goals
- A complete hardware-software co-design deployed on the Genesys-2 FPGA. This integrates the RISC-V-based VEGA soft-core processor to handle system control and data I/O.
- An accelerator synthesized on the FPGA fabric to offload heavy neural network computations from the VEGA core, ensuring real-time inference.

Design Contest Stage 1 Report

ii. Functional Specification

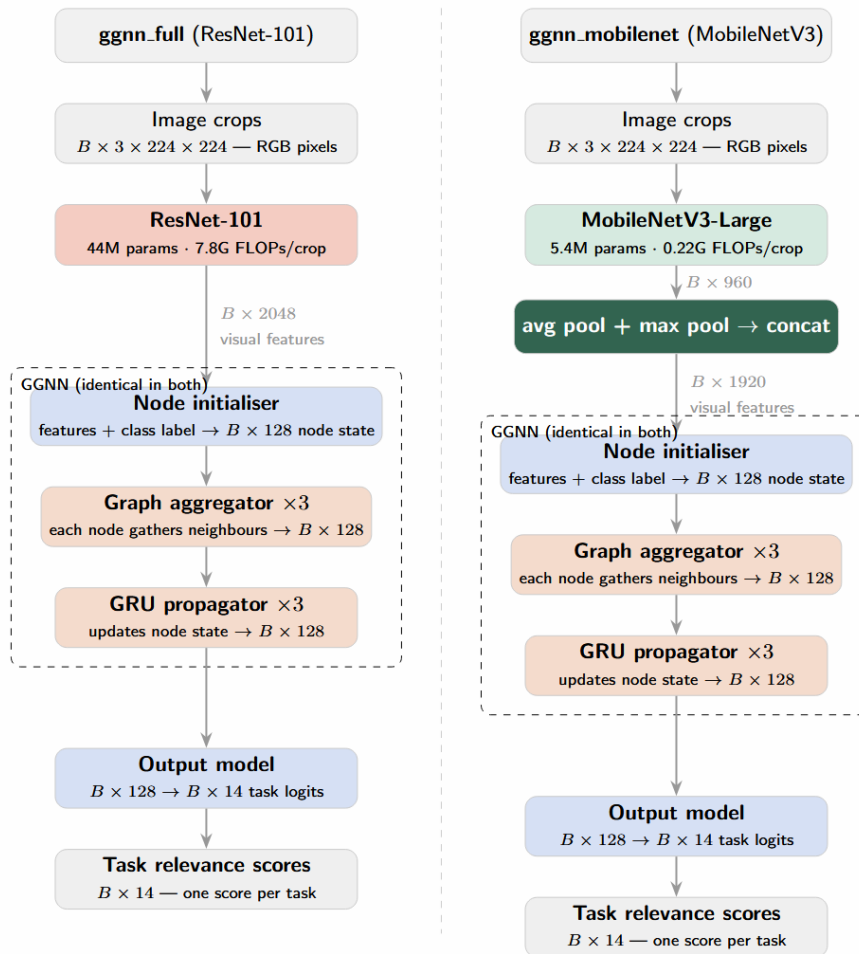


Figure 1: Pipelines of original Resnet101 based architecture and proposed MobileNetV3 based architecture

iii. Proposed Design

The proposed solution utilizes a heterogeneous System-on-Chip (SoC) hardware-software co-design strategy. By intelligently partitioning workloads between a general-purpose processor and a custom hardware accelerator, the system achieves real-time, task-driven object detection under strict edge computing constraints.

1. Hardware/Software Partitioning Strategy : Host System (VEGA RISC-V Soft-Core Processor): Synthesized on the Genesys-2 FPGA, the VEGA processor acts as the master controller. It handles non-compute-intensive operations: fetching image

Design Contest Stage 1 Report

data and task prompts (text) from external storage, image pre-processing, configuring Direct Memory Access transfers, and outputting the final selected bounding box.

To minimize instruction overhead during matrix operations, we propose a custom ISA extension inspired by RV-Matrix and RVV. This allows the VEGA core to dispatch complex tensor operations directly to the accelerator using single instructions, reducing the bottleneck of software-based loop management.

Custom Hardware Accelerator : The heavy neural network inferences specifically the visual feature extraction and the task-aware context modeling are offloaded to a custom-designed hardware accelerator mapped directly onto the FPGA's DSP slices and Block RAM (BRAM).

2. Feature Extraction Backbone: The MobileNet V3 Large Optimization

A critical optimization in this design is the replacement of the original CVPR 2019 framework's ResNet-101 backbone with a MobileNet V3 Large architecture.

Resource Efficiency: ResNet-101 contains roughly 44.5 million parameters and relies on standard, compute-heavy convolutions, making it highly impractical to deploy alongside a VEGA soft-core on a resource-constrained FPGA. MobileNet V3 Large reduces the parameter count to approximately 5.4 million (>85% reduction).

Hardware-Friendly Operations: MobileNet V3 Large leverages Depthwise Separable Convolutions, which factorize standard convolutions into a depthwise spatial convolution and a 1×1 pointwise convolution. This drastically reduces the required Multiply-Accumulate (MAC) operations, allowing the accelerator to achieve higher frame rates using fewer DSP slices.

3. Task-Aware Context Modeling (GGNN Integration)

The visual features extracted by the MobileNet V3 Large accelerator are combined with object class inputs and fed into the Gated Graph Neural Network (GGNN).

Systolic Array Implementation: The GGNN's core Gated Recurrent Unit (GRU) update rules heavily rely on dense matrix multiplications. These will be implemented as a customized, pipelined systolic array on the FPGA fabric.

4. Interconnect and Memory Architecture

To overcome the latency from fetching weights from external DDR3 memory, the design employs a Weight Stationary Dataflow.

Due to the compact size of MobileNet V3 Large, a significant portion of the network weights and intermediate feature maps can be cached directly in the FPGA's internal BRAM.

Design Contest Stage 1 Report

The VEGA processor, DDR3 memory controller, and custom neural network accelerator communicate via a high-bandwidth AXI4/AXI-Stream interconnect, ensuring seamless, low-latency data streaming during inference

iv. Analysis of Final Design

To achieve high-throughput inference for the Gated Graph Neural Network (GGNN), the architecture incorporates a 2D Systolic Array. This hardware structure facilitates a Weight-Stationary dataflow, where weights are buffered locally within the array to minimize data movement. This is specifically designed to handle the dense matrix-vector multiplications required by the GGNN's GRU update rules.

To address the "memory wall" and the high-latency bottleneck of fetching data from RAM, we implement a Matrix Tiling (Blocking) strategy.

Tiling: Large matrices are decomposed into smaller sub-blocks (tiles) that fit into the FPGA's local memory.

On-Chip Buffering: Instead of constant global memory access, we utilize Shared Scratchpad Memory (implemented via BRAM) to store tiles.

Parallel Access: This allows parallel access by different processing elements (PEs) within the systolic array, effectively mimicking the "shared memory" performance of high-end parallel processors but optimized for the Genesys-2 FPGA's specific resource constraints

Design Contest Stage 1 Report

v. Testing and Implementation of Final Design

We tested both models involving Resnet101 and MobileNetV3 as backbones separately.

Following images show the results.

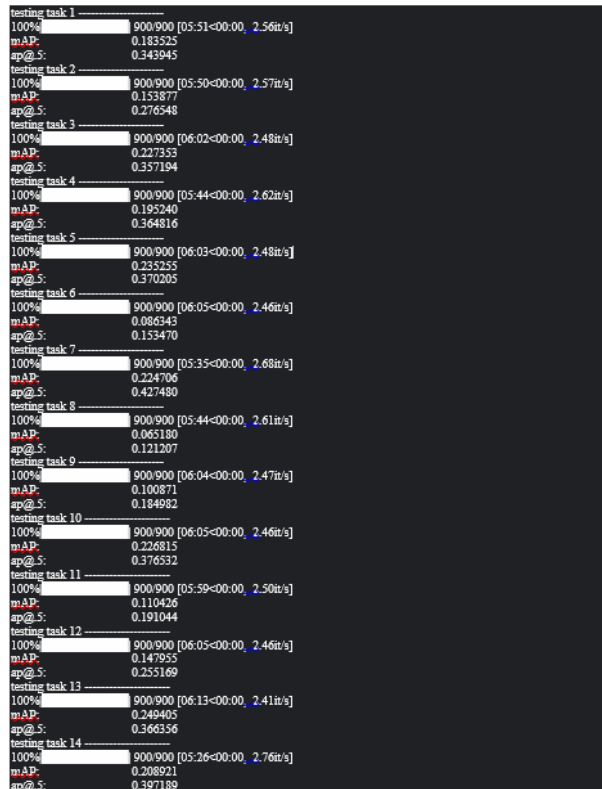


Figure 2: Original Implementation with R-CNN



Figure 3: Implementation using MobileNetV3 with R-CNN

	ResNet-101	MobileNetV3
Time for total inference	4966 seconds	2058 seconds
Average mAP	0.172562	0.127117
Parameter count	~44.5M parameters	5.4M parameters
FLOPs per crop	~7.8G FLOPs per crop	~0.22G FLOPs per crop
Model file size	~170MB	~13MB

Table 1: Comparison of Original Implementation with Implementation using MobileNetV3 with R-CNN

v. Results and Discussion

We replace the ResNet-101 backbone (~44.5M parameters, ~7.8G FLOPs per crop, 2048-d output) with MobileNetV3-Large (5.4M parameters, ~0.22G FLOPs per crop) using average+max pooling to produce 2048-d features that feed directly into the GGNN's initialiser, preserving the original single-projection design. With approximately 64 detections per image, ResNet-101 requires roughly 500G FLOPs per image at inference, while MobileNetV3-Large requires approximately 14G FLOPs, a **35× reduction in inference compute**. This directly translates to feasibility on the VEGA edge processor and Genesys-2 FPGA, where ResNet-101's 7.8G FLOPs per crop would be prohibitive for real-time deployment.

In terms of accuracy, ResNet-101 achieves approximately ~0.17 average mAP, while MobileNetV3-Large reaches ~0.13. This is a **comparable result achieved at a fraction of the computational budget**. The model size drops from approximately 170MB (ResNet-101) to 13MB (MobileNetV3-Large), an 13× reduction that is significant for edge deployment where on-chip memory is constrained. The trade-off is modest: tasks involving rare object categories score lower with MobileNet due to reduced feature capacity, but more common tasks demonstrate that pretrained MobileNetV3-Large features are highly discriminative for visually salient task categories. Overall, MobileNetV3-Large represents a favourable substitution for ResNet-101 in this pipeline

As future improvements we would like to propose an optimization path involves transitioning MobileNetV3 to **int8 quantization** and the GGNN to float16. This strategy aims to accelerate inference by replacing floating point calculations and enable the use of **four parallel sub-ALUs** within the FPGA fabric to process calculations concurrently. Such a quantized approach would further capitalize on the Genesys-2 FPGA's architecture, although its full realization was deferred due time constraints. Additionally, the pipeline's efficiency could be enhanced by replacing the AllLinearAggregator function with **k-nearest neighbors approach** for graph construction. By this restriction, the resulting adjacency matrices become sparse, creating significant room for specialized sparse matrix acceleration.

V. Conclusion

This project successfully demonstrates a highly efficient, edge-optimized framework for task-driven object detection, bridging the gap between semantic human intent and embedded hardware constraints. By utilizing a hardware-software co-design strategy on the Genesys-2 FPGA, the system leverages the indigenous 64-bit VEGA RISC-V processor for system control and custom Verilog accelerators for neural network execution.

A major contribution of this work is the algorithmic optimization of the feature extraction backbone. By replacing the computationally prohibitive ResNet-101 with MobileNetV3-Large, the design achieved a large reduction in inference compute and shrunk the model footprint. This aggressive optimization allowed the network to fit within the FPGA's internal BRAM while maintaining a highly competitive accuracy. Furthermore, accelerating the Gated Graph Neural Network (GGNN) via a custom 2D systolic array with matrix tiling and a Weight-Stationary dataflow effectively bypassed the von Neumann memory bottleneck.

VI. References

- Lin, T. Y., Maire, M., Belongie, S., Bourdev, L., Girshick, R., Hays, J., Perona, P., Ramanan, D., Zitnick, C. L., and Dollár, P. "Microsoft COCO: Common Objects in Context." *arXiv preprint arXiv:1405.0312*, 2014. <https://arxiv.org/abs/1405.0312>
- Sawatzky, J., Sourì, Y., Grund, C., and Gall, J. "What Object Should I Use? – Task Driven Object Detection." *arXiv preprint arXiv:1904.03000*, 2019. <https://arxiv.org/abs/1904.03000>
- Li, Y., Tarlow, D., Brockschmidt, M., and Zemel, R. "Gated Graph Sequence Neural Networks." *arXiv preprint arXiv:1511.05493*, 2015. <https://arxiv.org/abs/1511.05493>
- Yildirim, M., and Ozturk, O. "RISC-V Based TinyML Accelerator for Depthwise Separable Convolutions in Edge AI." *arXiv preprint arXiv:2511.21232*, 2025. <https://arxiv.org/abs/2511.21232>
- Xiang, L., Asudeh, O., Sabin, G., Sukumaran-Rajam, A., and Sadayappan, P. "cuTeSpMM: Accelerating Sparse-Dense Matrix Multiplication using GPU Tensor Cores." *arXiv preprint arXiv:2504.06443*, 2025. <https://arxiv.org/abs/2504.06443>

Design Contest Stage 1 Report

- Matsumi, W., and Mian, R. U. H. "Accelerating HDC-CNN Hybrid Models Using Custom Instructions on RISC-V GPUs." *arXiv preprint arXiv:2511.05053*, 2025.
<https://arxiv.org/pdf/2511.05053>
- H Logix & Solutions. "A Simple 2D Convolution Using Systolic Arrays." YouTube, 2020.
<https://youtu.be/WYw9guur0Lo>
- Gautam, T. "Must Know Technique in GPU Computing | Episode 4: Tiled Matrix Multiplication in CUDA C." YouTube, 2024. <https://youtu.be/Q3GgbfGTnVc>
- When Maths Meet Coding. "Depthwise Separable Convolution | Complete Tensor Operations for MobileNet Architecture." YouTube, 2020. <https://youtu.be/vfCvmenkbZA>
- RISC-V International. "RISC-V Vector Extension Specification, Version 1.0." 2023.
<https://lists.riscv.org/g/tech-vector-ext/attachment/691/0/riscv-v-spec-1.0.pdf>